

10 LinkedIn Posts

Built to Make People Follow DEPLOY

Short, story-driven posts around production AI, RAG, realtime systems, evals, RLHF, agentic workflows and Forward Deployed Engineering.

10	Ready-to-post drafts
3-5	Hashtags max per post
1	Clear CTA: follow DEPLOY

Voice: builder-first, short sentences, one strong insight, no corporate filler. **Goal:** make readers think, 'these people actually build this stuff' - then follow DEPLOY.

The AI demo worked. Production didn't care.

The first version looked great.

It answered the prompt. The UI worked. Everyone in the room could see the potential.

Then real users arrived.

Now latency mattered. Ambiguous questions mattered. Cost mattered. A wrong answer that looked believable mattered.

That is the part of AI engineering most demos skip.

The hard work starts after the first successful response - when you have to make the system reliable enough that someone can actually depend on it.

That gap between **"it works"** and **"we can deploy this"** is where the interesting engineering begins.

Follow DEPLOY for real breakdowns of what changes when AI systems move from demos to production.

Visual idea: 'Demo -> Real Users -> Failure Modes -> Production' as a minimal coded flow.

02 SQL RAG

Our SQL was valid. The answer was wrong.

That was the failure that changed how we built our SQL RAG system.

A user asks: **"Which accounts performed best this quarter?"**

The model can write perfectly valid SQL.

But what does "best" mean? Revenue? Growth? Conversion? Activity?

The dangerous errors were not syntax errors. They were confident answers based on the wrong business interpretation.

So we stopped treating text-to-SQL as a query-generation problem and started treating it as a **business-understanding problem**.

That shift - plus better evals and tighter context - pushed the strongest internal benchmark into the ~95% range.

Follow DEPLOY if you want more production RAG lessons beyond vector search + prompts.

Visual idea: Natural-language question on the left, 'semantic gap' in the middle, business answer on the right.

A 2-second pause can make a smart AI feel dumb.

We learned this while building a realtime AI interviewer.

The first pipeline technically worked: speech in, transcription, reasoning, speech out.

But the conversation felt wrong.

Every small delay stacked on top of the next one. The candidate answered, waited, waited again - and suddenly the system felt less intelligent than it actually was.

That changed the optimization target.

We stopped asking only **"Is the response correct?"** and started asking **"Does this feel like one continuous conversation?"**

In realtime AI, latency is not just infrastructure.

It is part of the product's intelligence.

Follow DEPLOY for more engineering lessons from realtime AI systems.

Visual idea: Conversation timeline comparing stitched latency vs realtime response flow.

04 COST OPTIMIZATION

We stopped optimizing cost per LLM call.

Because it was the wrong metric.

A cheap model that gives the wrong answer, triggers a retry, and eventually needs human review is not actually cheap.

A strong model used for every tiny task is not efficient either.

So the metric became:

Cost per correct answer.

That changed everything - context size, model routing, caching, and where we spent expensive reasoning.

The best cost optimization was not simply choosing a cheaper model.

It was removing unnecessary uncertainty before the model had to reason.

Follow DEPLOY for practical AI cost optimization that does not trade away quality.

Visual idea: 'Cost per call' crossed out, replaced with 'Cost per correct answer' and two simple flow comparisons.

Evals improved the system more than another round of prompt tweaking.

Early on, we checked obvious things.

Did the query execute? Did the answer look reasonable? Did the chart render?

Then we realized a system can pass all three and still answer the wrong business question.

So the evaluation target changed to:

Did the user receive the expected final answer?

We started tracking failures by category - business interpretation, retrieval, time logic, filters, follow-up context, result quality.

Now when quality dropped, we knew where it dropped.

That turned evals from a testing task into an engineering roadmap.

Follow DEPLOY for eval patterns we use to make AI systems behave more like software.

Visual idea: A central benchmark score surrounded by failure classes feeding a release gate.

06 AGENTIC SYSTEMS

The best agent is sometimes no agent.

While mapping an agentic learning system, we ended up with 30+ possible capabilities.

The easy mistake would have been calling all of them agents.

But reminders, schedulers, assignment triggers, profile updates, dashboards and approval flows do not need model judgment.

Some workflows need normal software.

Some need AI with a human in the loop.

Only a small number deserve real autonomy.

Calling everything "agentic" makes the architecture more expensive, harder to evaluate, and harder to trust.

Good AI architecture starts by deciding where AI should **not** be used.

Follow DEPLOY for agentic AI architecture without the agent-for-everything hype.

Visual idea: Three columns - Automation / AI + Human / Autonomous - with decreasing capability count.

RLHF starts with fixing your feedback.

A thumbs-up button is not a training pipeline.

User feedback is noisy.

Someone can dislike a correct answer because it is too long. Another user may prefer a different business definition. A changed policy can make yesterday's "good" answer wrong today.

So we treated feedback as **evaluation data first and training data second**.

Reviewed failures became new eval cases. Repeated preferences changed prompts and policies. Retrieval failures changed context. Hard examples informed model routing.

Only curated, high-quality preference pairs should eventually become fine-tuning or preference-optimization data.

The hard part of RLHF is not collecting clicks.

It is creating trustworthy signal.

Follow DEPLOY for practical feedback loops, evals, and model alignment from real product work.

Visual idea: User feedback -> review -> evals -> improvements -> re-evaluation loop.

08 ENGINEERING PROCESS

We built the slow version first. On purpose.

In one realtime AI build, the first working pipeline was not the final architecture.

It was slower than we wanted.

But it gave us something more valuable than a diagram: a baseline.

We could measure where the delay actually came from. We could test the interaction early. We could compare model choices against a real system instead of assumptions.

Sometimes the fastest way to production is to build the version you already know you will replace.

Because a working baseline tells you **what deserves optimization**.

Without it, you can spend weeks making the wrong part faster.

Follow DEPLOY for more lessons on sequencing AI builds without overengineering the first version.

Visual idea: V1 baseline -> measured bottlenecks -> optimized V2, with one metric line underneath.

AI should not replace the teacher. It should make every teacher better.

That became the core idea while designing an agentic learning roadmap.

Teachers were spending time on homework review, planning, progress tracking, parent updates, and repeated admin - while still lacking a real-time picture of what a student had mastered.

The opportunity was not a giant autonomous tutor.

It was a system that gives every teacher better context before class, better signals after class, and better continuity between sessions.

The model can set a quality floor.

The teacher still sets the ceiling.

That is a much more useful way to think about AI in human-led services.

Follow DEPLOY for more case studies on using AI to amplify experts instead of blindly replacing them.

Visual idea: Teacher at the center with diagnostics, mastery, practice and parent insights around the workflow - no human photo needed.

10 WHY DEPLOY

Most companies do not need 20 agents. They need one workflow that works.

AI strategy often starts too big.

A list of agents. A platform diagram. A transformation roadmap with no clear first win.

We prefer the opposite.

Pick one painful workflow.

Find the baseline.

Build the working system.

Integrate it into the real product.

Measure what changed.

Then expand.

That is the thinking behind **DEPLOY**.

Less AI theatre. More production systems that solve something measurable.

Follow DEPLOY if you care about how AI actually gets discovered, built, integrated, evaluated, and shipped.

Visual idea: One workflow expanding outward after a measurable production win.